

Traversing a list in order, post order and reverse order

A **list** is one of the most common ways to organize information. It is simply a collection of items arranged in a specific order. We use lists every day, often without realizing it.

Examples include:

- A shopping list
- A list of phone contacts
- A queue of people waiting for service
- A playlist of songs

The important feature of a list is that each item has a position relative to the others. There is a first item, a second item, and so on.

Why Lists Matter

Lists help us:

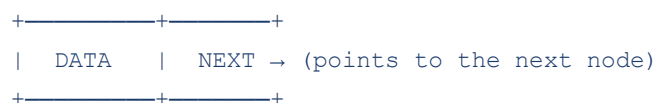
- Keep information organized
- Find information more easily
- Process items one at a time
- Maintain a sequence of activities or objects

What Is a Linked List?

A **linked list** is a way of storing a collection of items (numbers, names, anything) where each item holds two things:

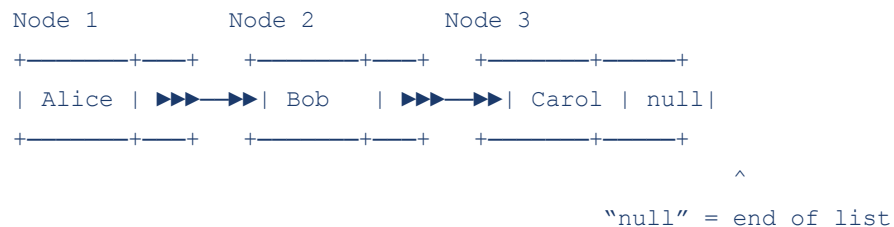
1. **The data itself** — the actual value you want to store (e.g., a name, a number).
2. A **link (pointer)** to the **next** item in the list.

Here is what a single node looks like drawn as a box:



Let's use a real example. Suppose we are storing the names of three friends: **Alice**, **Bob**, and **Carol**.

Each friend becomes one node:



Because each item “points” to the next, the items form a **chain**. That is why people often call it a linked list, the items are **linked together** like a chain.

The Problem with Arrays

Imagine you are saving seats for guests at a dinner party. You book a row of 5 seats right next to each other: seats 10, 11, 12, 13, and 14.

```
Seat:  | 10 | 11 | 12 | 13 | 14 |
Guest: | Anna | Bob | Cara | Dave |   |
```

This works fine... until someone wants to squeeze in **between** Anna and Bob. Now you have a problem: you must slide Bob, Cara, and Dave one seat to the right to make room. If you have 1,000 guests, sliding them all is very slow!

Similarly, removing a guest from the middle means every other guest must slide over to fill the gap. This “shuffling” is the main **weakness** of arrays.

How Linked Lists Solve This Problem

Linked lists take a completely different approach. Instead of seats in a row, imagine each guest sitting anywhere in a large hall and holding a sign that says “**The next guest is sitting at table #__.**”

If you want to add a new guest between Anna and Bob, you just:

1. Give the new guest (say, “Ed”) a seat anywhere in the hall.
2. Change Anna’s sign to point to Ed.
3. Give Ed a sign pointing to Bob.

Singly vs. Doubly Linked Lists (A Peek Ahead)

The type of linked list we have been describing so far — where each node only points **forward** to the next node — is called a **singly linked list**.

There is also a **doubly linked list** where each node has **two** pointers: one pointing forward (to the next node) and one pointing backward (to the previous node). This allows you to walk through the list in **either** direction.

Singly: Alice ►►► Bob ►►► Carol

Doubly: Alice ◄◄◄► Bob ◄◄◄► Carol

For the rest of these notes, we will focus on the simpler singly linked list.

Traversing a Linked List

Traversal simply means **visiting every node** in the list, one at a time, to read or process its data. Think of it as reading every page of a book from front to back (or back to front).

We will explore three ways to traverse a linked list:

1. **In-order (Forward)** — From the head (first node) to the tail (last node).
2. **Reverse-order (Backward)** — From the tail back to the head.
3. **Recursive (Post-order)** — Using a clever self-calling technique.

Forward Traversal (Head to Tail)

This is the most natural direction — just follow the arrows from start to finish.

Step-by-Step: Visiting Alice → Bob → Carol

Our list: Alice → Bob → Carol → (end)

Step 1: Start at the HEAD.

We look at the first node. The data says “Alice.” We note it down (or print it, or whatever we need to do).

```
HEAD
|
v
[ Alice | ► ] → [ Bob | ► ] → [ Carol | null ]
^
We are HERE
```

Step 2: Follow the pointer to the next node.

Alice’s NEXT pointer leads us to Bob. We note Bob’s name.

```
[ Alice | ► ] → [ Bob | ► ] → [ Carol | null ]
                  ^
                We are HERE
```

Step 3: Follow Bob’s pointer to Carol.

Bob’s NEXT pointer leads us to Carol. We note Carol’s name.

```
[ Alice | ► ] → [ Bob | ► ] → [ Carol | null ]
                                   ^
                                We are HERE
```

Step 4: Carol’s pointer is null.

This tells us we have reached the end of the list. We stop.

Result: We visited (and could print) the names in order: Alice, Bob, Carol.

Reverse-Order Traversal (Tail to Head)

Reverse traversal means visiting nodes in the **opposite** order: Carol → Bob → Alice.

Here is the tricky part: in a singly linked list, arrows only point **forward**. There are no backward arrows. So how do we go backwards?

There are two main strategies:

Strategy A: Use a Temporary Storage List (Stack)

Think of a stack as a pile of plates. The last plate you put on the pile is the first one you take off. Computer scientists call this “Last In, First Out” (**LIFO**).

Step 1: Walk forward through the list and push each node’s data onto a stack.

```
Forward walk: Alice → Bob → Carol
```

```
Stack (pile of plates, last on = first off):
```

```
+-----+
| Carol | ← top (last added)
+-----+
|  Bob  |
+-----+
| Alice | ← bottom (first added)
+-----+
```

Step 2: Pop (remove) items from the top of the stack one by one.

```
Pop Carol → Print "Carol"
Pop Bob   → Print "Bob"
Pop Alice → Print "Alice"
```

Result: Carol, Bob, Alice — the list in reverse order!

Recursive Traversal (Post-Order)

Recursion is a fascinating computer science concept where a process calls itself to solve a smaller version of the same problem. It sounds mysterious at first, but a real-life analogy makes it clear.

How Recursive Traversal Works on Alice → Bob → Carol

A recursive function does two things:

1. If the current node is null (end of list), stop and return.
2. Otherwise, call itself on the NEXT node (go deeper), then print the current node's data.

Walking into the list (going deeper):

```
Visit Alice → call on Bob → call on Carol → call on null (STOP)
```

Unwinding (coming back out, printing on the way):

```
null reached → return  
Print Carol → return to Bob  
Print Bob → return to Alice  
Print Alice → return to caller
```

Output: Carol, Bob, Alice — reverse order! This is called **post-order** because we print **after** we have visited all the nodes deeper in the list.

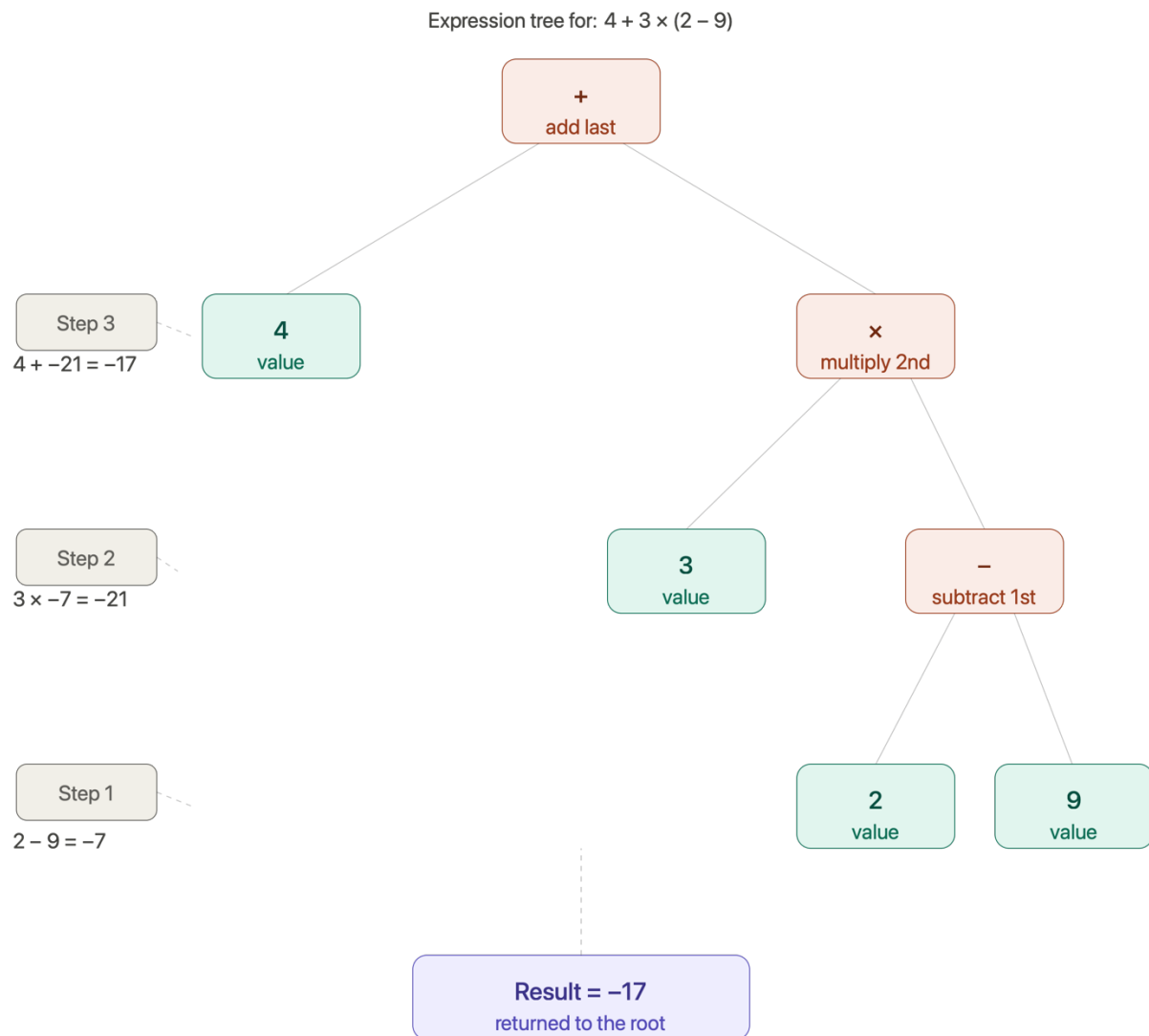
Side-by-Side Comparison of All Three Methods

Feature	Forward / In-Order	Reverse (Stack)	Recursive (Post-Order)
Direction of output	Alice → Bob → Carol	Carol → Bob → Alice	Carol → Bob → Alice
Extra memory needed?	No	Yes (stack storage)	Yes (call stack)
Simplicity	Easiest	Moderate	Elegant but tricky
Real-world use	Printing a list	Undo operations	Tree-based navigation

Example Calculator

Let me walk you through it step by step.

When we type $4 + 3 \times (2 - 9)$, a calculator doesn't just read left to right, it has to understand the *rules of mathematics* (brackets first, then multiply, then add). Here's how it works, step by step.



The calculator secretly builds a tree then traverses it bottom-up.

Here's the story in plain English:

Phase 1 Reading (building the tree)

The calculator reads $4 + 3 * (2 - 9)$ and, rather than immediately calculating, it first builds an "expression tree" like the one above. The tree captures the *order of operations* automatically:

- The brackets $(2 - 9)$ sit deepest — those leaves are at the bottom
- The $\times$ sits above them — it can't run until its children finish
- The $+$ sits at the very top (the root) — it runs last

Phase 2 Traversal (evaluating bottom-up)

This is where linked-list-style traversal kicks in, specifically, **post-order recursive traversal** (the kind we covered in the lecture notes!). The rule is: *visit children first, parent last*.

- **Step 1:** The calculator dives all the way to the deepest node: $2 - 9$. It has no children, so it calculates immediately: -7
- **Step 2:** It backs up to $\times$, which now has both its children resolved (3 and -7): $3 \times -7 = -21$
- **Step 3:** It backs up to $+$, the root, which now has both children (4 and -21): $4 + -21 = -17$

Why not just go left to right?

If the calculator read straight across $4 + 3 = 7$, then $7 \times 2 = 14$, then $14 - 9 = 5$ it would get **5**, which is wrong. The tree structure enforces the correct order automatically, without the calculator needing to "know" the BODMAS/PEMDAS rules separately. The *shape* of the tree is the rule.

This is essentially the same post-order recursion from the lecture: dive to the deepest node, compute on the way back up, and return the result to the parent, exactly like opening Russian dolls and reporting back what you found as you close each one.

Example 2:

Let's solve this equation:

$$14 - 19 + 10 / 2 + 3 * (12 - 9)$$

Step 1: Build an Expression Tree

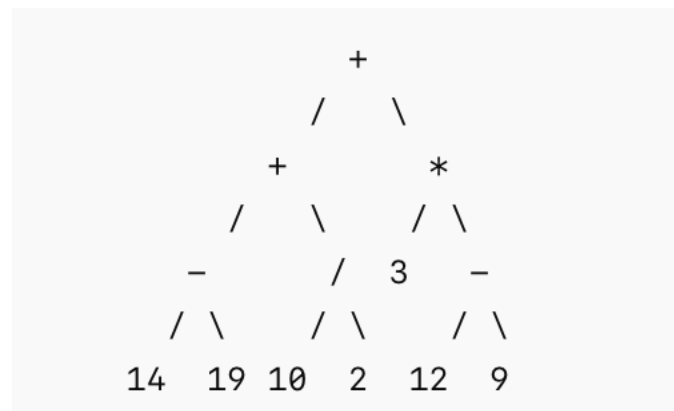
A calculator first organizes the expression according to operator precedence:

1. Parentheses first
2. Multiplication and division
3. Addition and subtraction

The expression becomes:

$$((14 - 19) + (10 / 2)) + (3 * (12 - 9))$$

The expression tree looks like:



In-Order Traversal

Rule

Visit:

Left → Root → Right

Let's traverse the tree.

Walkthrough

$14 - 19 + 10 / 2 + 3 * 12 - 9$

With parentheses:

$((14 - 19) + (10 / 2)) + (3 * (12 - 9))$

Result

In-order traversal reproduces the original mathematical expression.

Why Useful?

This is how calculators or compilers can display the expression in human-readable form.

Pre-Order Traversal

Rule

Visit:

Root $\rightarrow$ Left $\rightarrow$ Right

Traversal:

$+ + - 14 19 / 10 2 * 3 - 12 9$

This is called **Prefix Notation**.

Example

$+ + - 14 19 / 10 2 * 3 - 12 9$

The operator appears before its operands.

Post-Order Traversal

Rule

Visit:

Left → Right → Root

Traversal:

14 19 - 10 2 / + 3 12 9 - * +

This is called **Postfix Notation** (Reverse Polish Notation).

Why Calculators Love Post-Order Traversal

Suppose we process:

14 19 - 10 2 / + 3 12 9 - * +

using a stack.

Read 14

Stack: [14]

Read 19

Stack: [14, 19]

Read -

Take top two numbers:

14 - 19 = -5

Push result.

Stack: [-5]

Read 10

Stack: [-5, 10]

Read 2

Stack: [-5, 10, 2]

Read /

$10 / 2 = 5$

Push result.

Stack: [-5, 5]

Read +

$-5 + 5 = 0$

Push result.

Stack: [0]

Read 3

Stack: [0, 3]

Read 12

Stack: [0, 3, 12]

Read 9

Stack: [0, 3, 12, 9]

Read -

$12 - 9 = 3$

Push result.

Stack: [0, 3, 3]

Read *

$$3 * 3 = 9$$

Push result.

Stack: [0, 9]

Read +

$$0 + 9 = 9$$

Push result.

Stack: [9]

Final Answer

$$14 - 19 + 10 / 2 + 3 * (12 - 9) = 9$$

Connection to Traversal

Think of the expression tree as:

